

0.74.2. Glisser plus vite : glisse_croix à 3 — doublé, deux positions

Deux lettres, la même croix : deux appels, dans la même image.

1. Ce qu'il faut comprendre

C'est le 0.74 et le 0.74.1 réunis : deux appels, un par lettre.

Deux lutins, deux numéros (0 et 1) : ils glissent ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1629 octets** de cartouche.

1. UNE VARIABLE — PX

```
uint8_t px = 76;    // le A, en pixels
```

2. UNE VARIABLE — PY

```
uint8_t py = 68;
```

3. UNE VARIABLE — PXB

```
uint8_t pxb = 20;    // le B, en pixels
```

4. UNE VARIABLE — PYB

```
uint8_t pyb = 20;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

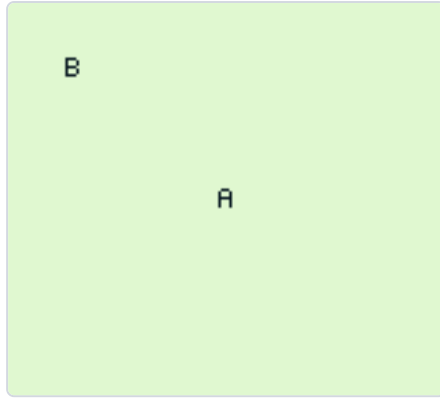
```
int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[0], 3);
        glisse_croix(1, pxb, pyb, ALPHABET[1], 3);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le A et le B suivent la croix ensemble.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les deux : 60 pixels

Ensuite : **0.75. Arrivé en (0, 0), le A devient B** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++